FORWARD-DEPLOYED ENGAGEMENT · COLABERRY PLATFORM · 6 OCT TO 10 NOV 2026

Ellis

The Colaberry platform is run by more than a hundred scheduled AI agents, and it can look healthy while it is not. Uptime checks say the process is up; they do not say whether each agent did its work. I was brought in to build the agent for the Platform Automation & Reliability role, working directly with its manager, K.D., under A.M.'s governance. Ellis tells a person, with evidence, what is healthy, what is broken, and what it cannot see. It reads the real platform's code and records and changes nothing live, and every capability counted here was verified by the platform itself, not by me.

40 / 48 capabilities verified by the platform

913 tests passing

7 decision records

184 commits

0 production writes

Source Command Center Artifacts

THE PROBLEM

A platform of a hundred agents that reports health it has not measured

From reading the platform's own source while building Ellis, each with file and line references in the repository.

- **Silence reads as health.** A security director stayed "normal" because no threat arrived from a scanner that never ran.
- **Failures are recorded as successes.** A wrapper shared by 104 agents logged a failed run as "no issues".
- **Records say work happened when it did not.** A bug-fix agent told reviewers "Auto-fix has been applied" when nothing was changed.
- **Controls that do not control.** Pause was ignored by five timers, so a paused agent kept running.
- **Scores that mean nothing.** An agent that has never run showed a health score of 100, in green.

The engagement came with its own boundaries, and they shaped everything after. The platform's repository is read-only to me: I never modify it. Every step up in Ellis's authority is a recorded human

decision: K.D. approves up to R2, A.M. from R3. And anything that changes production stays with a named person.

WHAT I BUILT

The evidence and judgement layer of an agent, built first and kept checkable

Ellis is an AI agent built in phases. This phase is the part a model's answers must later rest on: coded checks that answer one reliability question each, from the platform's records and pinned source, in one output shape.

Platform	Read boundary	39 capabilities	One output shape	People decide
Source code at a named commit, and dated record exports	No write method; every copied rule pinned to commit and file:line	One question each, from "did this job run?" to "who can reach this agent?"	Verdict, rows cited, blind spots named; severity only from K.D.'s table	K.D. and A.M. approve; a person makes every live change

Alongside: 16 higher-autonomy steps rehearsed on sandbox copies, and a shadow-mode scorer that judges nothing until K.D. sets its limits. Planned, not built: the governance gate, tool broker and model-backed agent core from K.D.'s architecture of 28 October.

Discovery came first: five stories (system classes, agent anatomy, the role and what transitions, the manager interview, three proposed tasks), reviewed with K.D. before building started: K.D. approved the first three on 21 and 22 October and cleared building on 26 October. After that the work ran one capability at a time, each committed with its story id so the platform could verify it from the repository, and each with a write-up in plain language. A.M. asked for the Capability Map on 10 November: one page per capability, its verified stories, and what it may not do yet.

MY CONTRIBUTION, VERSUS AI ASSISTANCE

Directed AI-assisted engineering, with the decisions on record

I set the scope and the rules, made or escalated the decisions, and decided what went to the stakeholders and when. Claude Code wrote most of the code and drafts under those rules: never touch the platform's repository, nothing committed or pushed without my explicit go, every claim checked against the repository before it is stated. The decisions that were not mine to make went to K.D. and A.M., and their rulings are recorded with dates.

What I cannot independently claim: the record of how closely I reviewed each change is mostly written in the same sessions that did the work. That is a real limit on the record, and I am leaving it visible.

PIECE OF WORK	WHO	EVIDENCE
Read-only boundary, repository rules, push only on my go	I DECIDED	Standing rules for the engagement; no command in Ellis can write to the platform
Five discovery stories	I DIRECTED, AI DRAFTED	Three approved by K.D. on 21 and 22 Oct; building cleared by K.D. on 26 Oct
The 39 capabilities and their tests	I DIRECTED, AI IMPLEMENTED	Each verified by the platform from a commit naming its story
Merging eight overlapping stories into existing checks	A.M. RULED, I BUILT	A.M.'s ruling of 4 Nov; recorded on each story as "Folded into"
Not re-committing a repeated R2 story	AI FLAGGED, I DECIDED	Its criterion was already verified on 26 Oct; raised with A.M. on 10 Nov instead
Shadow-mode scorer with no default limits	AI PROPOSED, I APPROVED	Commit b92b84d and decision record 007

KEY DECISIONS

Six real forks, with the rejected option on record

DECISION 001

Read-only first; change only in a sandbox

The platform's own repairs showed how a plausible change can be wrong, so every change Ellis makes runs on a sandbox copy.

Rejected: acting in production behind a feature flag; a dry-run mode in production

DECISION 002

Unknown is never healthy

Most of what was wrong in the platform was something reporting health it had not measured. Ellis lists what it cannot judge, with the reason.

Rejected: treating silence as healthy; hiding the parts that cannot be judged

DECISION 005

The evidence layer before the reasoning layer

A model's answer has to trace to records K.D. can check before anything is promoted, so the coded checks came first.

Rejected: starting with a model reading the records directly

DECISION 006

One output shape; severity only from K.D.'s table

K.D.'s architecture makes each check a tool. A tool that invents its own severity would be making K.D.'s decision.

Rejected: letting each tool or the model pick a severity; reporting every blind spot as unknown

DECISION 007

Shadow runs judged only against K.D.'s limits

The scorer counts misses and false alarms but judges nothing until K.D. sets the limits. An unknown on a real problem counts as a miss.

Rejected: a default limit "until K.D. decides"; leaving unknowns out of the miss rate

DECISION 004

Prove each key test can fail

Each key rule is broken on purpose and a named test must fail before the code is restored. A mutation nothing catches is a missing test.

Rejected: coverage figures as the bar

OBSTACLES

Three real ones, not smoothed over

Findings that overstated the problem

Colaberry Platform Reliability Audit · Ellis Trust and Risk Review

SITUATION

Reading the platform's code to build each capability turned up real defects, and I sent them to A.M. for review.

TASK

Keep every finding accurate enough to act on, because a finding that overstates costs the owners' trust in everything after it.

ACTION

A.M. reviewed 76 findings line by line, and 9 had to be rewritten. I changed the method: check the platform's own tests, baselines and documents before writing a finding, and label each one live, latent or a reporting gap. On 30 October K.D. and A.M. decided that defects are for Ellis to find, not for me to hunt, so the register closed and defects now appear only in Ellis's own output.

RESULT

61 of the 76 held as written or turned out worse. Five public platform pull requests fixed fourteen of them.

A green tick I could have forced, and did not

Emails to and from A.M., 1, 2 and 4 Nov · commit c97bff6

SITUATION

The platform added 81 rung stories to the board, but its plan, which the verifier reads, still carried only the original ids. Rung work could not verify.

TASK

Keep building without corrupting the record of what was verified.

ACTION

Renaming a rung commit to the base story id would have produced an instant green tick. I asked A.M. instead. He confirmed that naming each commit with its board id is right, that forcing the base id would overwrite a true result with a misleading one, and that the plan fix is platform work, affecting about 460 rung stories across four builders.

RESULT

All 16 sandbox rungs were built and committed under their true ids, unverified on the board until the plan carries them. The same rule held on 10 November, when a board story turned out to repeat work already verified.

Tests that passed for the wrong reason

Decision record 004 · docs/capabilities/agent-health-scoring.md · docs/shadow-evaluation.md

SITUATION

A test that passes the first time it runs may not test the rule it is named after.

TASK

Show, for every capability, that a wrong answer would actually be noticed.

ACTION

Each key rule is deliberately broken and a named test must fail. In agent health scoring, dropping a tie break went unnoticed at first, because the test data happened to satisfy it either way. In the shadow-mode scorer, removing the sort survived the first pass, because the idempotency test had only one miss. Both tests were strengthened until the mutation failed.

RESULT

Every capability write-up lists its mutations and how many tests each one fails. The scorer's 14 mutations are all caught.

TESTING AND RESULTS

What is verified, and by whom

Two different claims, kept apart. The platform verified 40 capability stories by reading this repository, which is checking by someone other than me. The 913 tests run on examples and fixtures built from the platform's source, not on production data.

40 / 48 capability stories verified by the platform	**913** tests passing, key rules shown to fail when broken	**16** higher-autonomy steps rehearsed on sandbox copies
25 live platform risks on record; 11 already fixed upstream	**14** findings fixed by 5 public platform pull requests	**0** writes to production

HONEST LIMITATIONS, AND WHAT IS NEXT

Named directly, not left for a reader to find

Nothing has run on live production records. All evidence so far is real platform code, tests and examples. The first live export waits on A.M.'s approval, and it is the next control before any capability moves up a level.

No approved autonomy level is on record. Built and verified is not approved: Ellis's own authority check resolves all 48 capabilities to none. Today Ellis answers and rehearses; it does not operate.

Shadow mode judges nothing yet. The scorer is built, but K.D.'s limits for misses and false alarms, and the severity table, are still empty. Both are K.D.'s to set.

The reasoning layer is not built. This phase makes no model calls at runtime. The gate, broker and model-backed agent core come next, under the same evidence contract, and two capabilities are held until they exist.

The sandbox is a seat belt, not a security boundary. Node's permission model stops trusted code making mistakes; it does not protect against malicious code. Anything untrusted would need a container or a separate machine.

Every figure on this page is in the repository: README.md, .colaberry/progress.json (the platform's verification record), docs/decisions/001 to 007, docs/capabilities/, docs/shadow-evaluation.md, and the Capability Map, Platform Reliability Audit and Trust and Risk Review in docs/artifacts/. As of 10 November 2026. Source: github.com/qninying/platform-reliability-agent.